

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №1
по дисциплине «Машинное обучение»
Тема: Автоматизированный выбор и настройка моделей машинного
обучения с использованием H2O AutoML.

Студент гр. 1306

Шаврин А.П.

Преподаватель

Куликов И.А.

Санкт-Петербург

2025

Цель работы.

Изучить принципы автоматизированного машинного обучения (AutoML) и освоить практические навыки применения системы H2O AutoML для автоматического подбора, обучения и оценки моделей машинного обучения при решении задач классификации и регрессии на реальных наборах данных.

Основные теоретические положения.

Современные задачи машинного обучения требуют подбора оптимального алгоритма и его гиперпараметров, что является трудоёмким и вычислительно затратным процессом. Подход **AutoML (Automated Machine Learning)** направлен на автоматизацию следующих этапов:

- предварительная обработка данных;
- выбор типа модели;
- настройка гиперпараметров;
- оценка качества моделей;
- формирование ансамбля лучших решений.

H2O AutoML - это программная платформа автоматизированного машинного обучения, реализованная на языке Java и предоставляющая Python-интерфейс. Система автоматически обучает и оценивает несколько типов моделей, включая:

- Gradient Boosting Machine (GBM);
- Random Forest;
- Generalized Linear Models (GLM);
- XGBoost;
- Deep Learning;
- ансамблевые модели (Stacked Ensembles).

H2O AutoML самостоятельно управляет вычислительным бюджетом, выполняя последовательный запуск моделей с различными конфигурациями и формируя **лидерборд** — упорядоченный список моделей по выбранной метрике качества.

На начальном этапе система выполняет:

- проверку и преобразование входных данных;
- автоматическое определение типов признаков;
- разбиение данных (при необходимости);
- вычисление базовой модели.

Затем в рамках заданного времени выполняется обучение различных моделей и их оценка. По завершении поиска формируется ансамбль лучших моделей, обеспечивающий наивысшее качество прогнозирования.

Постановка задачи.

В рамках лабораторной работы требуется исследовать возможности автоматизированного машинного обучения с использованием H2O AutoML при решении двух практических задач.

Задача 1. Классификация

Исследовать набор данных №32 из репозитория OpenML, содержащий данные для распознавания рукописных цифр. Необходимо:

- Загрузить набор данных.
- Разбить данные на обучающую и тестовую выборки.
- Использовать H2O AutoML для автоматического подбора моделей классификации.
- Оценить точность классификации на тестовых данных.
- Увеличить время работы AutoML и сравнить полученные результаты.
- Изменить количество используемых вычислительных ресурсов и проанализировать влияние на качество моделей.

Задача 2. Регрессия

Исследовать набор данных `concrete.csv`, содержащий информацию об образцах бетона. Цель — прогнозирование прочности на сжатие. Необходимо:

- Загрузить набор данных из файла.
- Разбить данные на обучающую и тестовую выборки.
- Использовать H2O AutoML для автоматического подбора регрессионных моделей.
- Оценить качество прогнозирования на тестовых данных.
- Вывести информацию о проделанном поиске и списке полученных моделей.
- Проанализировать две наилучшие модели и их вклад в итоговое решение.

Выполнение работы.

1. Распознавание рукописных цифр (OpenML dataset №32)

Для выполнения лабораторной работы была написана функция: `run_handwritten_digits_recognition_task`. Данная функция объединяет все этапы работы с набором данных: загрузку, разбиение, обучение моделей AutoML и оценку результатов.

Поскольку H2O — это самостоятельный сервер для выполнения моделей машинного обучения. Перед тем как загружать данные и обучать модели, необходимо его запустить, чтобы создать рабочую среду, в которой будут выполняться вычисления. Указание `nthreads` позволяет задействовать несколько потоков процессора для ускорения обучения моделей и параллельного выполнения задач, что особенно важно при работе с большими наборами данных или при обучении нескольких моделей одновременно.

Командой `h2o.init(nthreads=nthreads)` запускаем H2O, указываем количество потоков для параллельной работы. Результат запуска на рис 1.

```

Checking whether there is an H2O instance running at http://localhost:54321..... not found.
Attempting to start a local H2O server...
Java Version: openjdk version "17.0.17" 2025-10-21; OpenJDK Runtime Environment (build 17.0.17+10-Ubuntu-124.04); OpenJDK 64-Bit Server VM (build 17.0.17+10-Ubuntu-124.04, mixed mode, sharing)
Starting server from /home/aleksei/Development/LetiMaster/1_semester/AutoMachineLearning/lab1/.venv/lib/python3.10/site-packages/h2o/backend/bin/h2o.jar
Ice root: /tmp/tmpH7y9jC2h
JVM stdout: /tmp/tmpH7y9jC2h/h2o_aleksei_started_from_python.out
JVM stderr: /tmp/tmpH7y9jC2h/h2o_aleksei_started_from_python.err
Server is running at http://127.0.0.1:54321
Connecting to H2O server at http://127.0.0.1:54321 ... successful.
-----
H2O_cluster_uptime:      03 secs
H2O_cluster_timezone:    Etc/UTC
H2O_data_parsing_timezone: UTC
H2O_cluster_version:     3.46.0.9
H2O_cluster_version_age:  22 days
H2O_cluster_name:        H2O_from_python_aleksei_9f58b3
H2O_cluster_total_nodes: 1
H2O_cluster_free_memory: 1.403 GB
H2O_cluster_total_cores: 5
H2O_cluster_allowed_cores: 1
H2O_cluster_status:      Locked, healthy
H2O_connection_url:       http://127.0.0.1:54321
H2O_connection_proxy:     {"http": null, "https": null}
H2O_internal_security:    False
Python_version:           3.10.17 final
-----

```

Рисунок 1. Запуск H2O.

С помощью команд:

```
dataset = openml.datasets.get_dataset(dataset_id)
```

```
X, y, _, _ = dataset.get_data(target=dataset.default_target_attribute)
```

```
target_feature = "target"
```

```
df = pd.concat([X, y.rename(target_feature)], axis=1)
```

Загружаем набор данных №32 из OpenML. Объединяем признаки (X) и целевую переменную (y) в один DataFrame, поскольку это облегчит последующую конвертацию в формат H2O. Далее выводим основную информацию по набору рис 2.

```

RangeIndex: 10992 entries, 0 to 10991
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   input1      10992 non-null   float64
 1   input2      10992 non-null   float64
 2   input3      10992 non-null   float64
 3   input4      10992 non-null   float64
 4   input5      10992 non-null   float64
 5   input6      10992 non-null   float64
 6   input7      10992 non-null   float64
 7   input8      10992 non-null   float64
 8   input9      10992 non-null   float64
 9   input10     10992 non-null   float64
10  input11     10992 non-null   float64
11  input12     10992 non-null   float64
12  input13     10992 non-null   float64
13  input14     10992 non-null   float64
14  input15     10992 non-null   float64
15  input16     10992 non-null   float64
16  target      10992 non-null   category
dtypes: category(1), float64(16)

```

Рисунок 2. Информация по датасету.

Командой `train_df, test_df = train_test_split(df, test_size=test_size, random_state=seed, stratify=df[target_feature])` из библиотеки `sklearn` происходит разбиение датасета на обучающую и тренировочную выборку 20% на 80%. Стратифицированное разбиение (`stratify=df[target_feature]`) сохраняет пропорции классов, что важно для корректной оценки качества классификации

Далее данные переводятся в формат используемый H2O и помечаются как факторы для использования классификации:

```
train_h2o = h2o.H2OFrame(train_df)
test_h2o = h2o.H2OFrame(test_df)
features = [c for c in train_h2o.columns if c != target_feature]
train_h2o[target_feature] = train_h2o[target_feature].asfactor()
test_h2o[target_feature] = test_h2o[target_feature].asfactor()
```

H2O работает с собственным форматом `H2OFrame`, который оптимизирован для больших данных и параллельных вычислений. Приведение целевой переменной к категориальному типу (`asfactor`) необходимо, чтобы H2O понимал, что задача — классификация.

Для обучение моделей с использованием H2O AutoML используются команды:

```
aml = H2OAutoML(max_runtime_secs=300, seed=42, balance_classes=True)
aml.train(x=features, y="target", training_frame=train_h2o)
```

H2OAutoML автоматически обучает несколько моделей (GBM, Random Forest, Deep Learning, XGBoost и ансамбли) и подбирает гиперпараметры. `max_runtime_secs=300` ограничивает время поиска оптимальной модели до 5 минут. `balance_classes=True` улучшает качество классификации для редких цифр. Результат тренировочных моделей представлен на рисунке 3.

model_id	mean_per_class_error	logloss	rmse	mse
StackedEnsemble_BestOfFamily_1_AutoML_1_20251217_132002	0.0073371	0.0294966	0.0852142	0.00726146
GBM_1_AutoML_1_20251217_132002	0.00792963	0.0283863	0.0815356	0.00664805
XGBoost_1_AutoML_1_20251217_132002	0.01327	0.0510816	0.111449	0.0124208
GLM_1_AutoML_1_20251217_132002	0.064123	0.227893	0.248595	0.0617994
XGBoost_2_AutoML_1_20251217_132002	0.0970425	1.29153	0.712771	0.508042
[5 rows x 5 columns]				

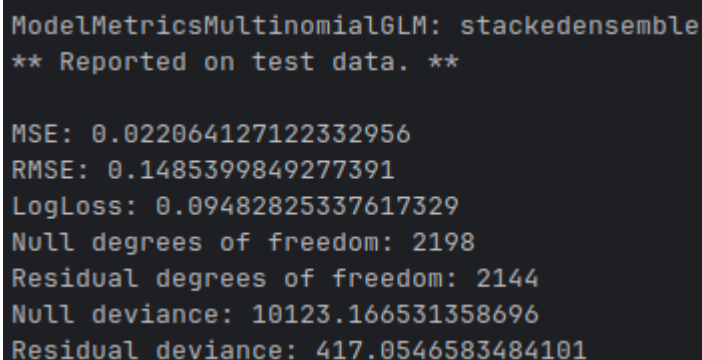
Рисунок 3. Результат тренировки моделей.

На тренировочных данных лидерборд H2O AutoML показывает, что лучшей моделью является StackedEnsemble_BestOfFamily, которая объединяет несколько моделей и обеспечивает минимальную ошибку классификации ($\text{mean_per_class_error} = 0.0073$). Модели GBM и XGBoost также демонстрируют высокую точность, в то время как GLM и отдельные слабые конфигурации XGBoost имеют заметно худшие показатели. Это подтверждает, что использование ансамблей повышает качество классификации.

Оценка точности на тестовых данных производится командой:

```
perf = aml.leader.model_performance(test_h2o)
print(perf)
```

Лучшая модель (leader) оценивается на тестовой выборке. Метрики точности, F1-score и logloss показывают реальное качество модели на новых данных. Результат команды представлен на рисунке 4.



```
ModelMetricsMultinomialGLM: stackedensemble
** Reported on test data. **

MSE: 0.022064127122332956
RMSE: 0.1485399849277391
LogLoss: 0.09482825337617329
Null degrees of freedom: 2198
Residual degrees of freedom: 2144
Null deviance: 10123.166531358696
Residual deviance: 417.0546583484101
```

Рисунок 4. Оценка точности на тестовых данных

На тренировочных данных модель StackedEnsemble показала очень низкие значения ошибок:

- $\text{mean_per_class_error} = 0.0073$
- $\text{logloss} = 0.0295$
- $\text{RMSE} = 0.0852$

Это говорит о том, что модель практически идеально подстроилась под тренировочные данные. На тестовых данных метрики немного выше:

- $\text{MSE} = 0.0221$
- $\text{LogLoss} = 0.0948$

- $RMSE = 0.1485$

Значительное уменьшение deviance по сравнению с нулевой моделью (Residual deviance = 417 против Null deviance = 10123) подтверждает, что модель хорошо обобщает на новые данные.

Таким образом модель обладает очень высокой точностью и незначительным переобучением, так как ошибки на тесте остаются низкими. Разница между метриками тренировочного и тестового наборов небольшая, что говорит о стабильности модели и её способности корректно предсказывать новые данные.

Из полученного объекта `model_performance` также можно получить Confusion Matrix (рисунок 5)

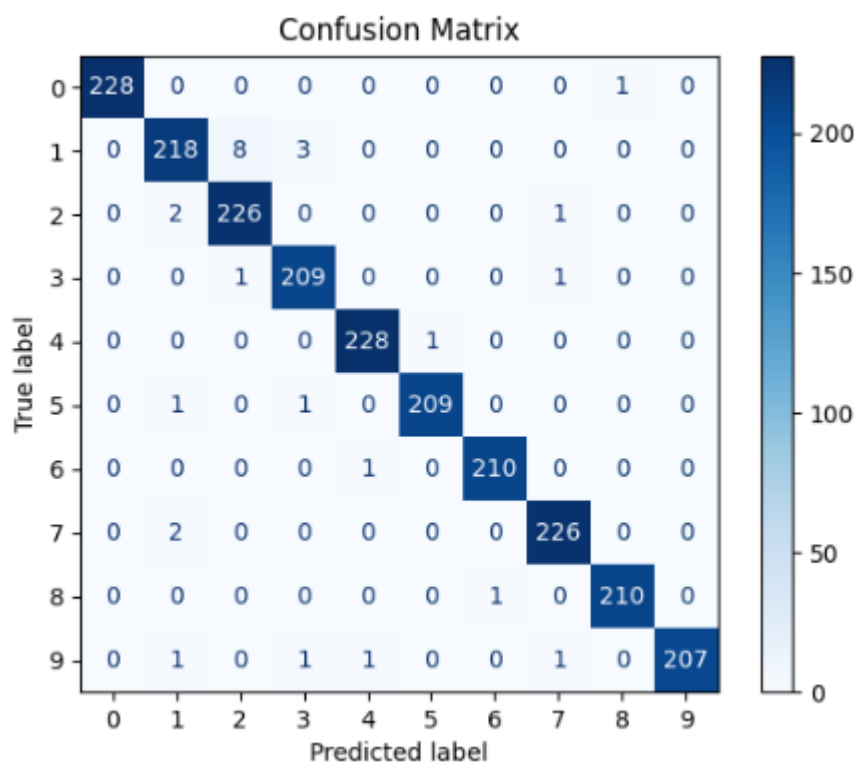


Рисунок 5. Confusion Matrix

Почти все цифры распознаны правильно, большинство значений сосредоточено на диагонали матрицы. Ошибки единичные, например, цифры 1 и 2 иногда путаются. Общий уровень ошибок крайне низок, что подтверждает эффективность модели в классификации всех классов.

Также есть возможность получить статистику по проценту правильных предсказаний в зависимости от числа предсказаний (рисунок 6)

```

Top-10 Hit Ratios:
k      hit_ratio
---  -
1      0.987267
2      0.997726
3      0.99909
4      0.99909
5      0.99909
6      0.999545
7      0.999545
8      0.999545
9      0.999545
10     1

```

Рисунок 6. Top-k Hit Ratio

Top-1 Hit Ratio = 0.987 показывает, что в 98.7% случаев модель сразу предсказывает правильный класс. Top-2 Hit Ratio ≈ 0.998 — правильный класс присутствует среди двух наиболее вероятных предсказаний почти всегда. Top-10 Hit Ratio = 1 означает, что правильный класс всегда входит в топ-10 предсказаний. Это демонстрирует высокую надежность и уверенность модели, особенно для практических приложений, где учитываются несколько возможных вариантов.

После число процессов было увеличено с 1 до 4х и получены следующие результаты (рисунок 7 - 10)

model_id	mean_per_class_error	logloss	rmse	mse
StackedEnsemble_AllModels_1_AutoML_1_20251217_134516	0.00646196	0.0293308	0.0824489	0.00679783
StackedEnsemble_BestOfFamily_1_AutoML_1_20251217_134516	0.0065531	0.0259857	0.07906	0.00625049
GBM_1_AutoML_1_20251217_134516	0.00666263	0.0249114	0.0734893	0.00540068
StackedEnsemble_BestOfFamily_2_AutoML_1_20251217_134516	0.00713645	0.0269536	0.0801702	0.00642726
DRF_1_AutoML_1_20251217_134516	0.0120864	0.140471	0.146347	0.0214175
GBM_2_AutoML_1_20251217_134516	0.0131439	0.140309	0.172089	0.0296145
XGBoost_1_AutoML_1_20251217_134516	0.0132698	0.0510243	0.111334	0.0123952
GBM_3_AutoML_1_20251217_134516	0.0136361	0.141783	0.173748	0.0301883
GBM_4_AutoML_1_20251217_134516	0.0144652	0.231111	0.235502	0.0554614
XGBoost_2_AutoML_1_20251217_134516	0.0188754	0.159426	0.194298	0.0377519
GLM_1_AutoML_1_20251217_134516	0.0481254	0.171477	0.208792	0.0435939

[11 rows x 5 columns]

Рисунок 7. Результат тренировки моделей (4 процесса).

```

ModelMetricsMultinomialGLM: stackedensemble
** Reported on test data. **

MSE: 0.05813963728561351
RMSE: 0.24112162343019655
LogLoss: 0.18331975138546805
Null degrees of freedom: 2198
Residual degrees of freedom: 1862
Null deviance: 10123.166531358696
Residual deviance: 806.2402665932888

```

Рисунок 8. Оценка точности на тестовых данных (4 процесса).

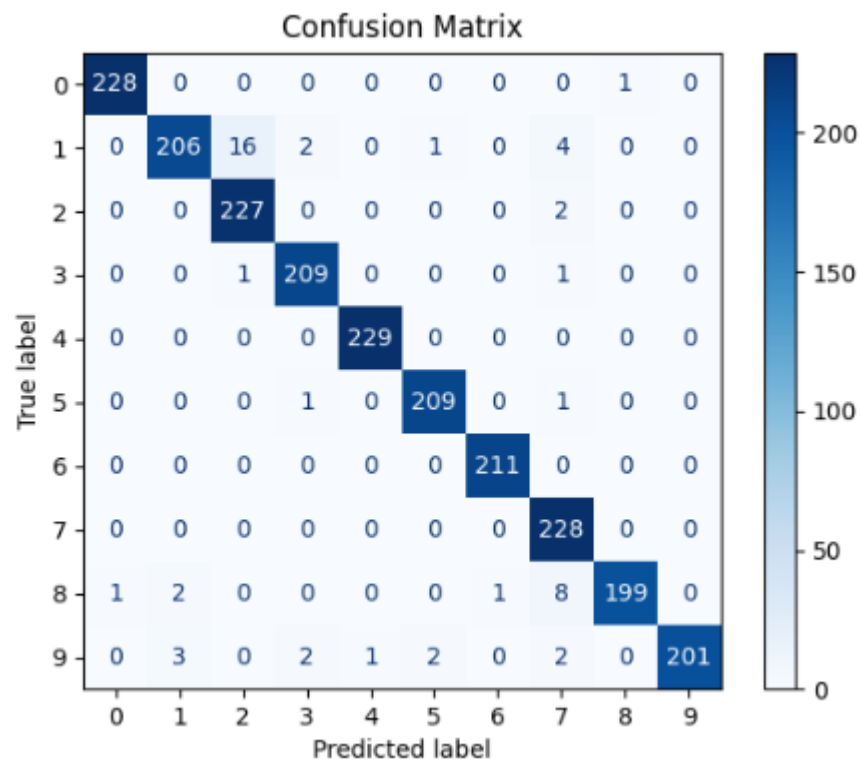


Рисунок 9. Confusion Matrix (4 процесса).

```

Top-10 Hit Ratios:
k    hit_ratio
---  -
1    0.976353
2    0.993633
3    0.998181
4    0.999545
5    1
6    1
7    1
8    1
9    1
10   1

```

Рисунок 10. Top-k Hit Ratio (4 процесса).

Как можно видеть из результатов экспериментов, использование разного числа потоков и времени работы AutoML оказывает влияние на качество моделей. В режиме с одним процессом и ограничением времени 300 секунд лучшая модель `StackedEnsemble_BestOfFamily` показала очень низкие ошибки на тренировочных данных ($\text{mean_per_class_error} \approx 0.0073$, $\text{RMSE} \approx 0.085$, $\text{LogLoss} \approx 0.0295$) и высокую точность на тестовой выборке ($\text{MSE} \approx 0.022$, $\text{RMSE} \approx 0.149$, $\text{LogLoss} \approx 0.095$). Матрица ошибок подтверждает, что почти все цифры классифицированы верно, а Top-1 Hit Ratio равный 0.987 и Top-2 Hit Ratio ≈ 0.998 демонстрируют высокую уверенность модели в предсказаниях.

При увеличении числа потоков до четырёх при том же лимите времени тренировочные ошибки ансамбля стали ещё ниже ($\text{mean_per_class_error} \approx 0.0065$, $\text{RMSE} \approx 0.079\text{--}0.082$, $\text{LogLoss} \approx 0.026\text{--}0.029$), что свидетельствует о более эффективном поиске и подборе моделей. Однако при этом метрики на тестовой выборке ухудшились ($\text{MSE} \approx 0.058$, $\text{RMSE} \approx 0.241$, $\text{LogLoss} \approx 0.183$), а ошибки в матрице классификации стали немного больше, что может указывать на частичное переобучение, вызванное более агрессивным поиском моделей и использованием большего числа параллельных процессов.

Таким образом, увеличение числа потоков позволило улучшить результаты на обучающих данных, но при этом наблюдается легкое снижение обобщающей способности модели на тестовой выборке, что подчёркивает необходимость баланса между временем обучения, параллельностью и контролем переобучения.

Теперь было проверено увеличение времени обработки с 300с до 600с (число потоков осталось тем же - 1). Результаты представлены на рисунках (11-15)

model_id	mean_per_class_error	logloss	rmse	mse
StackedEnsemble_BestOfFamily_2_AutoML_1_20251217_141754	0.00602445	0.0269478	0.0787903	0.00620791
StackedEnsemble_AllModels_1_AutoML_1_20251217_141754	0.0062158	0.0280853	0.0808335	0.00653405
StackedEnsemble_BestOfFamily_1_AutoML_1_20251217_141754	0.00632521	0.0256926	0.0778199	0.00605594
GBM_1_AutoML_1_20251217_141754	0.00654415	0.0245752	0.0720549	0.00519192
DRF_1_AutoML_1_20251217_141754	0.0104548	0.117588	0.140809	0.019827
XGBoost_2_AutoML_1_20251217_141754	0.0106006	0.0478623	0.107093	0.011469
GBM_2_AutoML_1_20251217_141754	0.0129982	0.100493	0.143401	0.0205637
XGBoost_1_AutoML_1_20251217_141754	0.0132698	0.0510243	0.111334	0.0123952
GBM_4_AutoML_1_20251217_141754	0.0145655	0.194105	0.20226	0.0409092
GBM_3_AutoML_1_20251217_141754	0.0152767	0.127347	0.160824	0.0258643
GLM_1_AutoML_1_20251217_141754	0.0451811	0.159713	0.200077	0.0400308

[11 rows x 5 columns]

Рисунок 11. Результат тренировки моделей (600 секунд).

```
ModelMetricsMultinomialGLM: stackedensemble
** Reported on test data. **

MSE: 0.02279157144859016
RMSE: 0.15096877640290446
LogLoss: 0.10244613646885306
Null degrees of freedom: 2198
Residual degrees of freedom: 2027
Null deviance: 10123.166531358696
Residual deviance: 450.5581081900157
```

Рисунок 12. Оценка точности на тестовых данных (600 секунд).

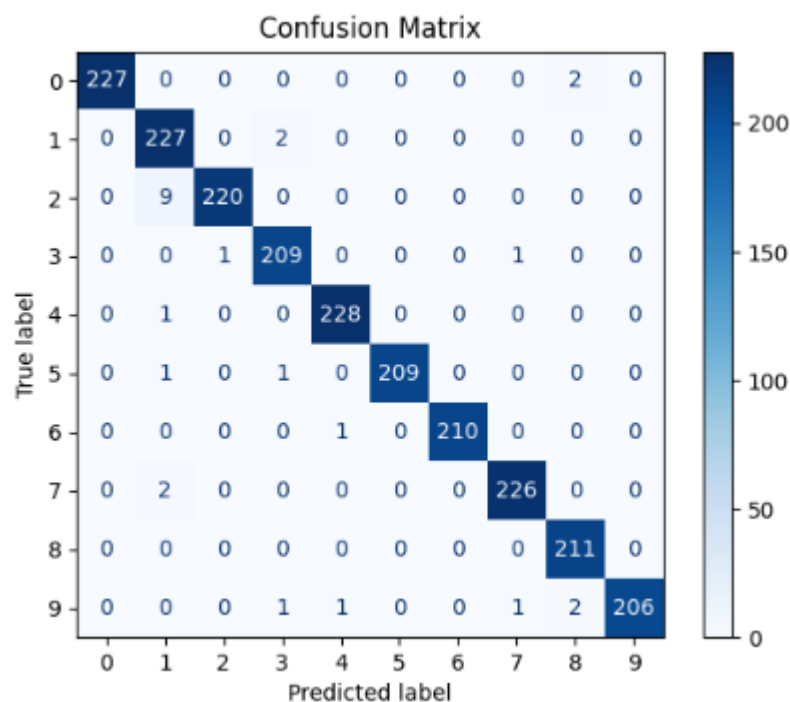
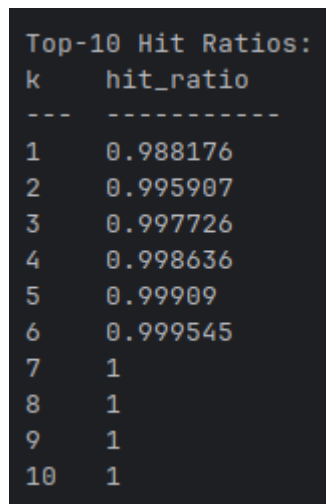


Рисунок 13. Confusion Matrix (600 секунд).



Top-10 Hit Ratios:	
k	hit_ratio
1	0.988176
2	0.995907
3	0.997726
4	0.998636
5	0.99909
6	0.999545
7	1
8	1
9	1
10	1

Рисунок 14. Top-k Hit Ratio (600 секунд).

Сравнивая результаты для одного потока при ограничении времени 300 с и 600 с, можно сделать следующие выводы. Увеличение времени поиска позволило AutoML исследовать больше моделей и конфигураций, что привело к небольшому улучшению показателей: значение MSE на тестовых данных осталось почти неизменным (≈ 0.0221 при 300 с и ≈ 0.0228 при 600 с), RMSE чуть вырос ($0.1485 \rightarrow 0.151$), а LogLoss немного увеличился ($0.0948 \rightarrow 0.102$).

По лидерборду видно, что ансамбли стали более разнообразными и включают новые комбинации моделей (StackedEnsemble_BestOfFamily_2, StackedEnsemble_AllModels_1), а отдельные модели GBM и XGBoost показали небольшие улучшения по RMSE и MSE.

Матрица ошибок демонстрирует, что число неправильно классифицированных объектов в отдельных классах снизилось или осталось на уровне предыдущего эксперимента, а Top-10 Hit Ratios показывает рост точности первых прогнозов (hit_ratio_1 повысился с 0.9873 до 0.9882).

Таким образом, увеличение времени поиска на одном потоке позволяет AutoML более тщательно исследовать пространство моделей, слегка улучшая точность предсказаний и создавая более качественный ансамбль без существенного изменения метрик MSE и RMSE на тестовых данных.

2. Прогнозирование прочности бетона (concrete.csv)

Для выполнения лабораторной работы была написана функция: `run_concrete_strength_task`. Данная функция объединяет все этапы работы с набором данных: загрузку, разбиение, обучение моделей AutoML и оценку результатов.

С помощью команды `df = pd.read_csv(file_path)` осуществляется загрузка и первичный анализ данных (рисунок 15).

```
=== INFO ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1030 entries, 0 to 1029
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   cement                 1030 non-null   float64
1   slag                   1030 non-null   float64
2   ash                    1030 non-null   float64
3   water                  1030 non-null   float64
4   superplastic           1030 non-null   float64
5   coarseagg              1030 non-null   float64
6   fineagg                1030 non-null   float64
7   age                    1030 non-null   int64
8   strength               1030 non-null   float64
dtypes: float64(8), int64(1)
memory usage: 72.5 KB
None
```

Рисунок 15. Информация по датасету.

Разбиение на обучающую и тестовую выборки происходит аналогично заданию 1 (80% данных для обучения, 20% — для теста. Конвертация в `H2OFrame` для совместимости с H2O AutoML).

Обучение моделей с H2O AutoML выполняется командами:

```
aml = H2OAutoML(max_runtime_secs=100, seed=42)
aml.train(x=features, y=target_feature, training_frame=train_h2o)
```

Автоматический подбор регрессионных моделей (GBM, DRF, Deep Learning, Stacked Ensemble). Ограничение времени (`max_runtime_secs`) позволяет контролировать ресурс обучения. Результат тренировочных моделей представлен на рисунке 16.

model_id	rmse	mse	mae	rmsle	mean_residual_deviance
StackedEnsemble_AllModels_3_AutoML_1_20251217_135845	4.41243	19.4695	2.95979	0.136551	19.4695
StackedEnsemble_AllModels_4_AutoML_1_20251217_135845	4.416	19.5011	2.96264	0.136812	19.5011
StackedEnsemble_BestOfFamily_3_AutoML_1_20251217_135845	4.46159	19.9058	2.96831	0.138914	19.9058
StackedEnsemble_AllModels_2_AutoML_1_20251217_135845	4.46413	19.9284	2.95986	0.139121	19.9284
StackedEnsemble_AllModels_1_AutoML_1_20251217_135845	4.51486	20.3839	3.0071	0.141739	20.3839
StackedEnsemble_BestOfFamily_2_AutoML_1_20251217_135845	4.52448	20.4709	3.01602	0.142567	20.4709
GBM_5_AutoML_1_20251217_135845	4.60907	21.2435	3.11634	0.142635	21.2435
GBM_grid_1_AutoML_1_20251217_135845_model_1	4.65203	21.6414	3.22988	0.146046	21.6414
GBM_grid_1_AutoML_1_20251217_135845_model_2	4.67288	21.8359	3.17201	0.146864	21.8359
GBM_4_AutoML_1_20251217_135845	4.688	21.9773	3.22069	0.14634	21.9773
StackedEnsemble_BestOfFamily_1_AutoML_1_20251217_135845	4.69401	22.0337	3.20474	0.155978	22.0337
XGBoost_1_AutoML_1_20251217_135845	4.71258	22.2084	3.20567	0.158536	22.2084
GBM_2_AutoML_1_20251217_135845	4.72434	22.3194	3.23348	0.149299	22.3194
GBM_3_AutoML_1_20251217_135845	4.72659	22.3406	3.24949	0.149977	22.3406
XGBoost_2_AutoML_1_20251217_135845	4.9789	24.7894	3.27838	0.156341	24.7894
XGBoost_grid_1_AutoML_1_20251217_135845_model_4	4.98744	24.8746	3.61934	0.167545	24.8746
XGBoost_3_AutoML_1_20251217_135845	4.99145	24.9146	3.32401	0.159176	24.9146
XGBoost_grid_1_AutoML_1_20251217_135845_model_3	5.17401	26.7704	3.387	0.161725	26.7704
XGBoost_grid_1_AutoML_1_20251217_135845_model_5	5.18413	26.8752	3.43594	0.165854	26.8752
XGBoost_grid_1_AutoML_1_20251217_135845_model_2	5.52164	30.4885	3.63851	0.170725	30.4885
XRT_1_AutoML_1_20251217_135845	5.78119	33.4222	4.30709	0.19887	33.4222
DRF_1_AutoML_1_20251217_135845	5.89837	34.7908	4.30079	0.204371	34.7908
XGBoost_grid_1_AutoML_1_20251217_135845_model_1	6.29674	39.6489	4.26672	0.199191	39.6489
XGBoost_grid_1_AutoML_1_20251217_135845_model_6	6.47529	41.9294	4.86648	0.207601	41.9294
GBM_1_AutoML_1_20251217_135845	6.58994	43.4273	5.13027	0.215128	43.4273
GBM_grid_1_AutoML_1_20251217_135845_model_3	6.61483	43.7559	4.85243	0.22653	43.7559
DeepLearning_1_AutoML_1_20251217_135845	7.86127	61.7995	5.93952	0.258419	61.7995
DeepLearning_grid_1_AutoML_1_20251217_135845_model_1	7.86165	61.8056	6.0234	0.245563	61.8056
DeepLearning_grid_3_AutoML_1_20251217_135845_model_1	8.42213	70.9323	6.43938	0.257376	70.9323
DeepLearning_grid_2_AutoML_1_20251217_135845_model_1	8.89792	79.173	6.8932	0.274434	79.173
DeepLearning_grid_1_AutoML_1_20251217_135845_model_2	9.97347	99.47	7.90089	0.309045	99.47
GLM_1_AutoML_1_20251217_135845	10.4532	109.269	8.24321	0.335583	109.269
GBM_grid_1_AutoML_1_20251217_135845_model_4	15.7392	247.722	12.6946	0.521148	247.722

[33 rows x 6 columns]

Рисунок 16. Результат тренировки моделей

Результаты тренировки регрессионных моделей показывают, что наилучшие показатели достигнуты ансамблевыми моделями StackedEnsemble_AllModels и StackedEnsemble_BestOfFamily с минимальными значениями RMSE (≈ 4.41 – 4.46), MSE (≈ 19.47 – 19.91) и MAE (≈ 2.96 – 2.97). Отдельные модели GBM, XGBoost и DeepLearning демонстрируют значительно более высокие ошибки, что указывает на меньшую точность предсказаний. Это подтверждает эффективность использования ансамблей, объединяющих разные алгоритмы, для получения более точных и устойчивых прогнозов регрессионной задачи.

Затем была проверена точность лидирующего ансамбля на тестовых данных и представлена на рисунке 17.

```
ModelMetricsRegressionGLM: stackedensemble
** Reported on test data. **

MSE: 18.917739100829387
RMSE: 4.349452735785203
MAE: 2.7798497071620787
RMSLE: 0.12733996056789124
Mean Residual Deviance: 18.917739100829387
R^2: 0.9338860660700835
Null degrees of freedom: 205
Residual degrees of freedom: 198
Null deviance: 58958.658023183045
Residual deviance: 3897.0542547708537
AIC: 1208.2632877604326
```

Рисунок 17. Оценка точности на тестовых данных

По метрикам на тестовых данных видно, что лучшая регрессионная модель демонстрирует высокую точность предсказаний. Значения ошибок остаются низкими: $MSE \approx 18.92$, $RMSE \approx 4.35$, $MAE \approx 2.78$ и $RMSLE \approx 0.127$, что сопоставимо с результатами на тренировочных данных ($RMSE \approx 4.41$ – 4.46 , $MSE \approx 19.47$ – 19.91 , $MAE \approx 2.96$ – 2.97). Коэффициент детерминации $R^2 = 0.934$ подтверждает, что модель объясняет большую часть вариации целевой переменной. Значительное снижение deviance по сравнению с нулевой моделью (Residual deviance ≈ 3897 против Null deviance ≈ 58958) также указывает на высокую предсказательную способность модели. В целом, сравнение с тренировочными метриками показывает, что ансамблевая модель сохраняет устойчивость и высокую точность на новых данных, демонстрируя минимальное переобучение.

Что бы получить информацию по метаобучателю необходимо выполнить команду `aml.leader.metalearner()` Это даст возможность получить информацию по метрикам точности на тренировочном датасете, а также на валидационном который представлен на рисунке 18.


```

ModelMetricsRegressionGLM: glm
** Reported on cross-validation data. **

MSE: 19.46954782094301
RMSE: 4.412431055658888
MAE: 2.95978701063821
RMSLE: 0.13655097510616823
Mean Residual Deviance: 19.46954782094301
R^2: 0.9297040025506844
Null degrees of freedom: 823
Residual degrees of freedom: 816
Null deviance: 228426.9468658697
Residual deviance: 16042.90740445704
AIC: 4802.744416766447

```

Рисунок 18. Оценка точности на валидационных данных для метаобучателя

На валидационных данных модель показала показатели, близкие к результатам, представленным в лидерборде: $MSE \approx 19.47$, $RMSE \approx 4.41$, $MAE \approx 2.96$, $RMSLE \approx 0.137$ и $R^2 \approx 0.93$. Это подтверждает, что модель устойчиво предсказывает новые данные и сохраняет высокую точность вне тренировочного набора.

Для получения вклада простых моделей в состав ансамбля можно из метаобучателя получить командой `aml.leader.metalearner().coef()` (рисунок 19)

	model	weight
1	GBM_5_AutoML_1_20251217_135845	0.301700
2	GBM_grid_1_AutoML_1_20251217_135845_model_1	0.247986
5	XGBoost_1_AutoML_1_20251217_135845	0.224163
3	GBM_grid_1_AutoML_1_20251217_135845_model_2	0.165695
19	GBM_grid_1_AutoML_1_20251217_135845_model_3	0.072958
9	XGBoost_grid_1_AutoML_1_20251217_135845_model_4	0.018933
20	DeepLearning_1_AutoML_1_20251217_135845	0.005750
12	XGBoost_grid_1_AutoML_1_20251217_135845_model_5	0.000000
15	DRF_1_AutoML_1_20251217_135845	0.000000
23	GLM_1_AutoML_1_20251217_135845	0.000000
22	DeepLearning_grid_1_AutoML_1_20251217_135845_m...	0.000000
21	DeepLearning_grid_1_AutoML_1_20251217_135845_m...	0.000000
18	GBM_1_AutoML_1_20251217_135845	0.000000
17	XGBoost_grid_1_AutoML_1_20251217_135845_model_6	0.000000
16	XGBoost_grid_1_AutoML_1_20251217_135845_model_1	0.000000
24	GBM_grid_1_AutoML_1_20251217_135845_model_4	0.000000
14	XRT_1_AutoML_1_20251217_135845	0.000000
13	XGBoost_grid_1_AutoML_1_20251217_135845_model_2	0.000000
11	XGBoost_grid_1_AutoML_1_20251217_135845_model_3	0.000000
10	XGBoost_3_AutoML_1_20251217_135845	0.000000
8	XGBoost_2_AutoML_1_20251217_135845	0.000000
7	GBM_3_AutoML_1_20251217_135845	0.000000
6	GBM_2_AutoML_1_20251217_135845	0.000000
4	GBM_4_AutoML_1_20251217_135845	0.000000
0	Intercept	-1.396917

Рисунок 19. Информация о классификаторах и их коэффициентах

Анализ состава мета-ансамбля показывает, что в объединение вошли преимущественно модели GBM и XGBoost, которые получили наибольшие веса. Наиболее значимый вклад внесла модель GBM_5 с весом 0.302, затем GBM_grid_1_model_1 с весом 0.248 и XGBoost_1 с весом 0.224. Остальные модели имеют меньшие веса (0.166 и ниже), а часть моделей вообще не использована (вес = 0). Таким образом текущий ансамбль формируется за счет нескольких сильных градиентных бустинговых моделей, которые вносят основной вклад в предсказания, а остальные модели служат вспомогательными или были проигнорированы при обучении мета-ансамбля. Такой подход позволяет объединять сильные предсказатели, повышая стабильность и точность итоговой модели.

Выводы.

В ходе лабораторной работы были изучены принципы автоматизированного машинного обучения (AutoML) и освоены практические навыки применения системы H2O AutoML для решения задач классификации и регрессии на реальных данных. На примере набора данных рукописных цифр (OpenML №32) была выполнена классификация, включающая загрузку и подготовку данных, разбиение на обучающую и тестовую выборки, автоматический подбор и обучение различных моделей, включая GBM, XGBoost, GLM, Deep Learning и ансамбли StackedEnsemble. Проведён анализ влияния времени обучения и количества потоков на качество моделей: увеличение времени с 300 с до 600 с позволило чуть улучшить точность предсказаний, а увеличение числа потоков ускорило поиск моделей (позволило рассмотреть большее число за то же время), но привело к лёгкому переобучению. Оценка качества моделей выполнялась с использованием метрик MSE, RMSE, LogLoss, mean_per_class_error, матрицы ошибок и Top-k Hit Ratios, что позволило выявить лидерские ансамбли с высокой точностью и устойчивостью к новым данным.

В задаче регрессии на наборе `concrete.csv` была реализована автоматическая генерация моделей для прогнозирования прочности бетона. Лучшие результаты показали ансамблевые модели `StackedEnsemble`, объединяющие преимущественно GBM и XGBoost, с минимальными ошибками ($RMSE \approx 4.4$, $MSE \approx 19.5$, $MAE \approx 2.96$) и высоким коэффициентом детерминации ($R^2 \approx 0.93$). Анализ вклада простых моделей в мета-ансамбль показал, что основные предсказания обеспечивают несколько сильных градиентных бустинговых моделей, а остальные модели играют вспомогательную роль или не были задействованы.

Таким образом, работа позволила изучить полный цикл применения AutoML: от подготовки данных до автоматического подбора моделей и формирования ансамблей, а также оценить влияние вычислительных ресурсов и времени обучения на качество и обобщающую способность моделей. Полученные навыки демонстрируют практическую ценность AutoML для быстрого и эффективного решения задач классификации и регрессии.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

main.py

```
import openml
import h2o
from h2o.automl import H2OAutoML
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    ConfusionMatrixDisplay
)

def run_handwritten_digits_recognition_task(
    dataset_id=32,
    max_runtime=600,
    nthreads=1,
    test_size=0.2,
    seed=42
):
    # Инициализация H2O
    h2o.init(nthreads=nthreads)

    # Загрузка данных
    dataset = openml.datasets.get_dataset(dataset_id)
    X, y, _ = dataset.get_data(target=dataset.default_target_attribute)
    target_feature = "target"
    df = pd.concat([X, y.rename(target_feature)], axis=1)

    print("=== INFO ===")
    print(df.info())
    print("\n=== HEAD ===")
    print(df.head(10))
    print(f"\n=== STATISTIC ===")
    print(df.describe())

    # Разбиение на train/test
    train_df, test_df = train_test_split(
        df, test_size=test_size, random_state=seed,
        stratify=df[target_feature]
    )

    train_h2o = h2o.H2OFrame(train_df)
    test_h2o = h2o.H2OFrame(test_df)

    features = [c for c in train_h2o.columns if c != target_feature]

    # Используем классификацию
    train_h2o[target_feature] = train_h2o[target_feature].asfactor()
    test_h2o[target_feature] = test_h2o[target_feature].asfactor()
```

```

# AutoML
aml = H2OAutoML(
    max_runtime_secs=max_runtime,
    seed=seed,
    balance_classes=True
)

aml.train(
    x=features,
    y=target_feature,
    training_frame=train_h2o
)

# Результат тренировки моделей
lb = aml.leaderboard
print(lb.head(rows=lb.nrows))

perf = aml.leader.model_performance(test_h2o)
print(perf)
cm = perf.confusion_matrix().as_data_frame()
print("\nConfusion Matrix:")
print(cm)

cm_values = cm.iloc[:10, :10].values.astype(int)
labels = cm.columns[:10]
disp = ConfusionMatrixDisplay(confusion_matrix=cm_values,
display_labels=labels)
disp.plot(cmap="Blues", values_format="d")
plt.title("Confusion Matrix")
plt.savefig("ConfusionMatrix(HandwrittenDigits).png")

def run_concrete_strength_task(
    file_path="concrete.xls",
    max_runtime=100,
    nthreads=1,
    test_size=0.2,
    seed=42,
    target_feature="strength"
):
    # Инициализация H2O
    h2o.init(nthreads=nthreads)

    # Загрузка данных
    df = pd.read_csv(file_path)
    print("=== INFO ===")
    print(df.info())
    print("\n=== HEAD ===")
    print(df.head(10))
    print(f"\n=== STATISTIC ===")
    print(df.describe())

    # Разбиение на train/test
    train_df, test_df = train_test_split(
        df, test_size=test_size, random_state=seed
    )

```

```

train_h2o = h2o.H2OFrame(train_df)
test_h2o = h2o.H2OFrame(test_df)
features = [c for c in df.columns if c != target_feature]

# AutoML
aml = H2OAutoML(
    max_runtime_secs=max_runtime,
    seed=seed,
)

aml.train(
    x=features,
    y=target_feature,
    training_frame=train_h2o
)

# Результат тренировки моделей
lb = aml.leaderboard
print(lb.head(rows=lb.nrows))

perf = aml.leader.model_performance(test_h2o)
print(perf)

leader = aml.leader
print(leader.model_id)

metalearner = leader.metalearner()
print(metalearner)

coefs = metalearner.coef()

coef_df = (
    pd.DataFrame.from_dict(coefs, orient="index", columns=["weight"])
    .reset_index()
    .rename(columns={"index": "model"})
)
coef_df = coef_df.sort_values("weight", ascending=False)
print(coef_df)

if __name__ == "__main__":
    run_handwritten_digits_recognition_task()

    # run_concrete_strength_task()

```